

NOOL

Database & Table Design Document

nool-core backend — PostgreSQL schema reference

Scope: All 40 application tables, services/core

Source of truth: SQLAlchemy ORM models,
`src/domain/models/`

Migration tool: Alembic, `database/migrations/`

Generated: 2026-08-30

Contents

1. Part I — Database Design Document

- Overview & Technology
- Design Principles & Conventions
- Shared Enumerations
- Domain Map & Entity Relationships
- Multi-tenancy Model

2. Part II — Table Design Document

- Foundation / Organization Structure (6 tables)
- Roster (4 tables)
- Curriculum (5 tables)
- Voice Tests & Assessment (6 tables)
- Custom Questions & Datasets (4 tables)
- Homework (6 tables)
- Question Papers (9 tables)
- Retest / Improvement (3 tables)
- Leaderboard (1 table)
- Reporting (2 tables)
- Support (2 tables)
- Import Jobs (1 table)
- AI Assistant (1 table)

Database Design Document

1. Overview & Technology

nool's backend (`nool-core`) persists all application state in a single PostgreSQL 16 database, accessed exclusively through SQLAlchemy 2.0's async ORM from the FastAPI service. There is one schema shared by every tenant (school); isolation between schools is enforced in application code via a `school_id` foreign key on every tenant-scoped table, not by separate databases or Postgres schemas per tenant.

40

TABLES

13

DOMAIN AREAS

100%UUID PRIMARY
KEYS**0**ORM
RELATIONSHIP()
LINKS

2. Design Principles & Conventions

UUID primary keys everywhere

Every table's primary key is a `UUID` generated client-side via `uuid.uuid4()` (the shared `IdMixin` in `mixins.py`), never a database auto-increment integer. This keeps IDs non-guessable and safe to generate before a row is committed (used for cross-referencing objects created in the same request).

Explicit foreign keys, no ORM relationship() graph

Every cross-table link is a plain `ForeignKey` column. The codebase deliberately does not declare SQLAlchemy `relationship()` attributes anywhere — all joins are written explicitly at the query/repository layer. This trades ORM convenience (no lazy-loading, no `.children` auto-fetch) for predictable, visible query plans and avoids N+1 surprises.

Server-side timestamps

Tables that need a creation timestamp use the shared `TimestampMixin`, which sets `created_at` via Postgres `now()` as a `server_default` — not application-clock time — so the value is consistent regardless of which app server or timezone wrote the row.

1:1 relationships via shared/foreign primary key

Where a table is strictly one-to-one with another, its primary key is the parent's own foreign key rather than a separate surrogate id: `student_profiles.user_id`, `student_points.student_id`, `subscriptions.school_id` (unique, not PK, since `subscriptions` keeps its own id for the FK from `plans`), and `question_paper_validation.paper_id`.

Enums for closed sets, JSONB for open/flexible sets

Fixed, small vocabularies (status, role, bloom level, question type) are native Postgres enum types. Variable-shape or admin-configurable data (a plan's `enabled_features`, a report's `metrics/filters`, a

school's `default_bloom_distribution`) is stored as JSONB instead of being modeled as further child tables.

Junction tables carry the composite **UNIQUE** constraint, not the **PK**

Many-to-many and one-to-many "attribute" tables (e.g. `voice_test_bloom_levels`, `question_paper_bloom_distribution`, `grade_subjects`) keep their own UUID PK but enforce true uniqueness with a composite **UNIQUE** constraint on the natural key (e.g. `(test_id, bloom_level)`), so the PK stays a stable identity, not a proxy for uniqueness.

3. Shared Enumerations

These enum types recur across many tables rather than being redefined per-table.

bloom_level — Bloom's Taxonomy cognitive levels

- REMEMBER
- UNDERSTAND
- APPLY
- ANALYZE
- EVALUATE
- CREATE

Used on: voice test bloom levels/scores, custom questions, dataset questions, homework bloom distribution, question paper bloom distribution, AI assessor session bloom levels, retest bloom comparison.

role — `users.role`

- SUPER_ADMIN
- SCHOOL_ADMIN
- TEACHER
- STUDENT

user_status — reused for users, school grades, and classes

- PENDING
- ACTIVE
- DEACTIVATED

Note: `school_grades.status` and `classes.status` reuse this same enum type (ACTIVE/DEACTIVATED are the practically-used values there — a grade/section is "allocated" when ACTIVE).

school_status — `schools.status`

- ONBOARDING
- ACTIVE
- SUSPENDED

assignment_target_mode — who a Voice Test / Homework is assigned to

- WHOLE_CLASS
- SPECIFIC_STUDENTS

test_status — voice_tests.status

- DRAFT
- SCHEDULED
- ACTIVE
- COMPLETED
- RESULTS_PROCESSING
- RESULTS_READY

question_type

- MCQ
- SHORT_ANSWER
- LONG_ANSWER
- TRUE_FALSE

4. Domain Map & Entity Relationships

Tables group into thirteen domain areas. Arrows below read "has many" from left to right, following the actual foreign-key direction (child table holds the FK).

- **Foundation:** plans → schools → subscriptions / upgrade_requests; schools → users → audit_logs
- **Roster:** schools → school_grades → classes → teacher_class_assignments; classes → student_profiles (1:1 with users)
- **Curriculum:** subjects → chapters → topics; schools xsubjects → school_curriculum; school_grades xsubjects → grade_subjects
- **Voice Tests:** classes/subjects/chapters/topics → voice_tests → voice_test_bloom_levels / voice_test_target_students → student_test_results → student_test_result_bloom_scores; voice_tests/retest_attempts → ai_assessor_sessions → ai_assessor_session_bloom_levels
- **Custom Questions & Datasets:** schools/classes/subjects → custom_questions; datasets → dataset_questions; schools xdatasets → school_datasets
- **Homework:** voice_tests → homework → homework_datasets / homework_bloom_distribution / homework_target_students / homework_questions → student_homework_progress
- **Question Papers:** schools/subjects → question_papers → question_paper_dataset_shares / _chapters / _topics / _sections / _bloom_distribution / _difficulty_distribution / _validation (1:1) / _questions
- **Retest / Improvement:** homework → retest_attempts → retest_bloom_comparison; voice_tests + homework → topic_performance
- **Leaderboard:** users → student_points (1:1)
- **Reporting:** users/schools → report_configurations → report_shares
- **Support:** schools/users → support_tickets → support_ticket_comments
- **Import Jobs:** schools/users → import_jobs
- **AI Assistant:** users (teachers) → assistant_messages

5. Multi-tenancy Model

Every school-scoped table carries a direct or indirect `school_id`. Global, Super-Admin-owned catalog data (`subjects`, `chapters`, `topics`, `datasets`, `dataset_questions`, `plans`) has no `school_id` at all — it is shared across every tenant and only becomes school-specific through an explicit join table (`school_curriculum`, `grade_subjects`, `school_datasets`) that opts a school into a subset of the catalog. This is the same pattern used for the round-7 "Subject" (formerly "Curriculum") feature: Super Admin assigns from the global `subjects` catalog into a school via `school_curriculum/grade_subjects`; School Admin can only read its resulting assignment, never write it.

Enforcement of tenant isolation happens entirely in the API layer (every school-scoped query filters by `current_user.school_id`) — there is no Postgres row-level security policy. A missing `WHERE school_id = ...` clause in a new route is the main cross-tenant risk surface to review for in code review.

PART II

Table Design Document

Every table, in full column detail. **PK** primary key · **FK** foreign key · **UQ** part of a unique constraint.
Unless noted, primary keys are `id` `UUID` (default `uuid4()`).

A. Foundation / Organization Structure

plans

Subscription tiers a school can be placed on (feature/seat limits).

Column	Type	Null	Notes
id PK	UUID	no	default uuid4()
name	str	no	unique
price_label	str	no	
teacher_limit	int	no	
student_limit	int	no	
test_limit	int	yes	null = unlimited
question_paper_limit	int	yes	null = unlimited
enabled_features	JSONB (list[str])	yes	null = all Feature enum values enabled
active	bool	no	default true

schools

A tenant. One row per onboarded school.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	server default now()
name	str	no	
board	str	no	e.g. "CBSE"
city	str	no	
contact_email	str	no	
address	str	yes	
pincode	str	yes	added round 7
contact_phone	str	yes	
principal_name	str	yes	
status	enum school_status	no	default ONBOARDING
plan_id FK→plans.id	UUID	yes	deprecated — <code>subscriptions</code> is now the source of truth
default_bloom_distribution	JSONB (dict[str,int])	yes	
logo_data_uri	Text	yes	base64 data URI, shown in admin console sidebars

subscriptions

Current plan + billing status for one school. 1:1 with schools.

Column	Type	Null	Notes
id PK	UUID	no	
school_id FK→schools.id UQ	UUID	no	unique — enforces 1:1
plan_id FK→plans.id	UUID	no	
status	enum subscription_status	no	default TRIAL (TRIAL, ACTIVE, PAST_DUE, CANCELED)
renews_at	timestampz	no	

upgrade_requests

A School Admin's request to move to a higher plan; tracked to resolution by Super Admin.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
school_id FK→schools.id	UUID	no	
requested_by FK→users.id	UUID	no	
message	Text	yes	
status	enum upgrade_request_status	no	default PENDING (PENDING, CONTACTED, RESOLVED)
resolved_at	timestampz	yes	
resolved_by FK→users.id	UUID	yes	

users

Every human account across all four roles, in every school.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
firebase_uid	str	yes	unique; null until first login (invited-but-not-signed-in users)
email	str	no	unique
display_name	str	no	
role	enum role	no	SUPER_ADMIN, SCHOOL_ADMIN, TEACHER, STUDENT
school_id FK→schools.id	UUID	yes	null only for SUPER_ADMIN
status	enum user_status	no	default ACTIVE (PENDING, ACTIVE, DEACTIVATED)
phone_number	str	yes	
employee_id	str	yes	
must_change_password	bool	no	default false

audit_logs

Append-only log of admin actions, for traceability.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
actor_id FK→users.id	UUID	no	
action	str	no	
target_type	str	no	
target_id	str	no	
detail	Text	yes	

B. Roster

school_grades

A "Class" (e.g. Class 10) that a school has been allocated by Super Admin.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
school_id FK→schools.id UQ	UUID	no	unique with grade
grade UQ	int	no	1–12
status	enum user_status	no	default ACTIVE; ACTIVE = allocated/usable

UNIQUE(school_id, grade)

classes (model: SchoolClass)

A "Section" within a Class (e.g. Class 10 · A) — the unit students and teacher assignments actually belong to.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
school_id FK→schools.id UQ	UUID	no	
grade UQ	int	no	denormalized copy for convenience
section UQ	str	no	e.g. "A"
grade_id FK→school_grades.id	UUID	yes	
status	enum user_status	no	default ACTIVE

UNIQUE(school_id, grade, section)

teacher_class_assignments

Which teacher teaches which subject to which section.

Column	Type	Null	Notes
id PK	UUID	no	
teacher_id FK→users.id UQ	UUID	no	
class_id FK→classes.id UQ	UUID	no	
subject_id FK→subjects.id UQ	UUID	no	

UNIQUE(teacher_id, class_id, subject_id)

student_profiles

Student-specific fields, 1:1 extension of a STUDENT-role user.

Column	Type	Null	Notes
user_id PK FK→users.id	UUID	no	PK is the FK itself — enforces 1:1
class_id FK→classes.id UQ	UUID	no	
roll_number UQ	int	no	
guardian_name	str	yes	
guardian_phone	str	yes	
date_of_birth	Date	yes	

UNIQUE(class_id, roll_number)

C. Curriculum

subjects

Global subject catalog, owned by Super Admin, shared by every school.

Column	Type	Null	Notes
id PK	UUID	no	
name	str	no	unique

chapters

A chapter within a global subject.

Column	Type	Null	Notes
id PK	UUID	no	
subject_id FK→subjects.id	UUID	no	
name	str	no	

topics

A topic within a chapter — the finest curriculum granularity.

Column	Type	Null	Notes
id PK	UUID	no	
chapter_id FK→chapters.id	UUID	no	
name	str	no	

school_curriculum

Which subjects Super Admin has assigned to a given school (school-level, not per-grade). School Admin can read this but — since round 7 — can no longer toggle it.

Column	Type	Null	Notes
id PK	UUID	no	
school_id FK→schools.id UQ	UUID	no	
subject_id FK→subjects.id UQ	UUID	no	
enabled	bool	no	default true

UNIQUE(school_id, subject_id)

grade_subjects

Which subjects are assigned to a specific Class (grade) — the finer-grained assignment used by the Classes page's per-grade Subject toggle.

Column	Type	Null	Notes
id PK	UUID	no	
grade_id FK→school_grades.id UQ	UUID	no	
subject_id FK→subjects.id UQ	UUID	no	
enabled	bool	no	default true

UNIQUE(grade_id, subject_id)

D. Voice Tests & Assessment

voice_tests

A test a teacher assigns to a class on one chapter/topic, assessed across chosen Bloom levels.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
class_id FK→classes.id	UUID	no	
subject_id FK→subjects.id	UUID	no	
chapter_id FK→chapters.id	UUID	no	
topic_id FK→topics.id	UUID	yes	
duration_minutes	int	no	
completion_window_hours	int	no	
target_mode	enum assignment_target_mode	no	WHOLE_CLASS or SPECIFIC_STUDENTS
status	enum test_status	no	default DRAFT
assigned_count	int	no	default 0
completed_count	int	no	default 0

voice_test_bloom_levels

Which Bloom levels a given test assesses.

Column	Type	Null	Notes
id PK	UUID	no	
test_id FK→voice_tests.id UQ	UUID	no	
bloom_level UQ	enum bloom_level	no	

UNIQUE(test_id, bloom_level)

voice_test_target_students

The real per-student delivery list for a test, regardless of target_mode. **Round-7 fix:** previously only populated for SPECIFIC_STUDENTS; WHOLE_CLASS tests now resolve the class roster at creation time and populate this table too — every "my assigned tests" read path joins through here.

Column	Type	Null	Notes
id PK	UUID	no	
test_id FK→voice_tests.id UQ	UUID	no	
student_id FK→users.id UQ	UUID	no	

UNIQUE(test_id, student_id)

student_test_results

One student's overall result on one test.

Column	Type	Null	Notes
id PK	UUID	no	
test_id FK→voice_tests.id UQ	UUID	no	
student_id FK→users.id UQ	UUID	no	
mastery_percent	int	no	

UNIQUE(test_id, student_id)

student_test_result_bloom_scores

Per-Bloom-level breakdown of one student's result — feeds the Student app's Progress/report-card screen and the /me/progress "next best focus" computation.

Column	Type	Null	Notes
id PK	UUID	no	
student_test_result_id FK→student_test_results.id UQ	UUID	no	
bloom_level UQ	enum bloom_level	no	
percent	int	yes	

UNIQUE(student_test_result_id, bloom_level)

ai_assessor_sessions

One live voice-assessment session ("Milo") — either against a scheduled test or a retest attempt.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
student_id FK→users.id	UUID	no	
test_id FK→voice_tests.id	UUID	yes	exactly one of test_id/retest_attempt_id set
retest_attempt_id FK→retest_attempts.id	UUID	yes	
context_label	str	no	
duration_seconds	int	no	

ai_assessor_session_bloom_levels

Bloom levels covered within one AI assessor session.

Column	Type	Null	Notes
id PK	UUID	no	
session_id FK→ai_assessor_sessions.id UQ	UUID	no	
bloom_level UQ	enum bloom_level	no	

UNIQUE(session_id, bloom_level)

E. Custom Questions & Datasets

custom_questions

A school's own hand-authored questions — a school-owned counterpart to the global dataset catalog. Shown to the school under Datasets (round 7) as well as under Question Bank.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
school_id FK→schools.id	UUID	no	
class_id FK→classes.id	UUID	yes	exactly one of class_id/grade set (CHECK constraint)
grade	int	yes	
subject_id FK→subjects.id	UUID	no	
chapter_id FK→chapters.id	UUID	no	
topic_id FK→topics.id	UUID	yes	
bloom_level	enum bloom_level	no	
question_type	enum question_type	no	
text	Text	no	
options	JSONB (list[str])	yes	MCQ options
answer	Text	no	
created_by FK→users.id	UUID	no	
collection_name	str	yes	groups questions into a named set; null = general bank

CHECK: exactly one of (class_id, grade) is non-null

datasets

Global (Super-Admin-owned) named question dataset catalog entry.

Column	Type	Null	Notes
id PK	UUID	no	
name	str	no	
question_count	int	no	
description	Text	no	
subject_id FK→subjects.id	UUID	yes	

dataset_questions

The actual questions inside a global dataset.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
dataset_id FK→datasets.id	UUID	no	
chapter_id FK→chapters.id	UUID	yes	
topic_id FK→topics.id	UUID	yes	
bloom_level	enum bloom_level	no	
question_type	enum question_type	no	
text	Text	no	
options	JSONB	yes	
answer	Text	no	
created_by FK→users.id	UUID	no	

school_datasets

Which global datasets Super Admin has enabled for a given school.

Column	Type	Null	Notes
id PK	UUID	no	
school_id FK→schools.id UQ	UUID	no	
dataset_id FK→datasets.id UQ	UUID	no	
enabled	bool	no	default true

UNIQUE(school_id, dataset_id)

F. Homework

homework

AI-generated remedial homework targeting a specific mastery gap found in a voice test.

Column	Type	Null	Notes
id PK	UUID	no	
test_id FK→voice_tests.id	UUID	no	
class_id FK→classes.id	UUID	no	
gap_topic	Text	no	
gap_mastery_percent	int	no	
total_questions	int	no	
difficulty	enum homework_difficulty	no	EASY, MIXED, HARD
target_mode	enum assignment_target_mode	no	
completion_window_hours	int	yes	
status	enum homework_status	no	default GENERATING (GENERATING, REVIEW, ASSIGNED, IN_PROGRESS, COMPLETED, FAILED)
assigned_count	int	no	default 0
completed_count	int	no	default 0

homework_datasets

Which datasets were drawn from to generate a homework's questions.

Column	Type	Null	Notes
id PK	UUID	no	
homework_id FK→homework.id UQ	UUID	no	
dataset_id FK→datasets.id UQ	UUID	no	

UNIQUE(homework_id, dataset_id)

homework_bloom_distribution

Target Bloom-level mix for a homework's generated questions.

Column	Type	Null	Notes
id PK	UUID	no	
homework_id FK→homework.id UQ	UUID	no	
bloom_level UQ	enum bloom_level	no	
value	int	no	

UNIQUE(homework_id, bloom_level)

homework_target_students

Per-student delivery list for a homework (same pattern as `voice_test_target_students`).

Column	Type	Null	Notes
id PK	UUID	no	
homework_id FK→homework.id UQ	UUID	no	
student_id FK→users.id UQ	UUID	no	

UNIQUE(homework_id, student_id)

homework_questions

The generated questions belonging to one homework, in delivery order.

Column	Type	Null	Notes
id PK	UUID	no	
homework_id FK→homework.id	UUID	no	
order	int	no	
bloom_level	enum bloom_level	no	
dataset_id FK→datasets.id	UUID	no	
text	Text	no	
answer	Text	no	

student_homework_progress

One student's progress against one homework.

Column	Type	Null	Notes
id PK	UUID	no	
homework_id FK→homework.id UQ	UUID	no	
student_id FK→users.id UQ	UUID	no	
status	enum student_homework_status	no	default ASSIGNED (ASSIGNED, IN_PROGRESS, COMPLETED)
qa_completed	bool	no	default false
confirmed_at	timestamp tz	yes	
expired	bool	no	default false

UNIQUE(homework_id, student_id)

G. Question Papers

question_papers

A generated exam paper — configuration plus generation/review lifecycle state.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
school_id FK→schools.id	UUID	no	
created_by FK→users.id	UUID	no	
name	str	no	
exam_type	str	no	
board	str	no	
grade	int	no	
language	str	no	
subject_id FK→subjects.id	UUID	no	
total_marks	int	yes	
duration_minutes	int	yes	
question_types	ARRAY(str)	yes	
instructions	ARRAY(str)	yes	
allow_internal_choice	bool	no	default true
avoid_duplicate_concepts	bool	no	default true
respect_chapter_weightage	bool	no	default true
avoid_recent_repetition	bool	no	default true
status	enum question_paper_status	no	default DRAFT (DRAFT, GENERATING, VALIDATION_FAILED, REVIEW, FINALIZED)
custom_header_text	str	yes	
logo_data_uri	Text	yes	

question_paper_dataset_shares

What % of a paper's questions should be drawn from each chosen dataset.

Column	Type	Null	Notes
id PK	UUID	no	
paper_id FK→question_papers.id UQ	UUID	no	
dataset_id FK→datasets.id UQ	UUID	no	
percent	int	no	

UNIQUE(paper_id, dataset_id)

question_paper_chapters

Which chapters a paper draws from.

Column	Type	Null	Notes
id PK	UUID	no	
paper_id FK →question_papers.id UQ	UUID	no	
chapter_id FK →chapters.id UQ	UUID	no	

UNIQUE(paper_id, chapter_id)

question_paper_topics

Which topics a paper draws from.

Column	Type	Null	Notes
id PK	UUID	no	
paper_id FK →question_papers.id UQ	UUID	no	
topic_id FK →topics.id UQ	UUID	no	

UNIQUE(paper_id, topic_id)

question_paper_sections

A named section of the paper (e.g. "Section A") with its question count/marks-each.

Column	Type	Null	Notes
id PK	UUID	no	
paper_id FK →question_papers.id	UUID	no	
label	str	no	
question_count	int	no	
marks_each	int	no	

question_paper_bloom_distribution

Target Bloom-level mix for the paper.

Column	Type	Null	Notes
id PK	UUID	no	
paper_id FK →question_papers.id UQ	UUID	no	
bloom_level UQ	enum bloom_level	no	
value	int	no	

UNIQUE(paper_id, bloom_level)

question_paper_difficulty_distribution

Target Easy/Medium/Hard mix for the paper.

Column	Type	Null	Notes
id PK	UUID	no	
paper_id FK→question_papers.id UQ	UUID	no	
level UQ	enum paper_difficulty_level	no	EASY, MEDIUM, HARD
value	int	no	

UNIQUE(paper_id, level)

question_paper_validation

Automated QA check results for a generated paper. 1:1 with question_papers.

Column	Type	Null	Notes
paper_id PK FK→question_papers.id	UUID	no	PK is the FK itself — enforces 1:1
coverage_percent	int	no	
marks_accounted_for	int	no	
duplicate_count	int	no	
quality_percent	int	no	
issues	ARRAY(str)	no	default []

question_paper_questions

The actual generated questions belonging to a paper, in order.

Column	Type	Null	Notes
id PK	UUID	no	
paper_id FK→question_papers.id	UUID	no	
order	int	no	
bloom_level	enum bloom_level	no	
marks	int	no	
text	Text	no	

H. Retest / Improvement

retest_attempts

A student's retest against a homework, tracking baseline vs. retest mastery.

Column	Type	Null	Notes
id PK	UUID	no	
homework_id FK→homework.id UQ	UUID	no	
student_id FK→users.id UQ	UUID	no	
status	enum student_retest_status	no	default ASSIGNED (ASSIGNED, IN_PROGRESS, COMPLETED, RESULT_READY)
attempts_used	int	no	default 0
baseline_percent	int	yes	
retest_percent	int	yes	
improvement_percent	int	yes	
completed_at	timestampz	yes	

UNIQUE(homework_id, student_id)

retest_bloom_comparison

Before/after Bloom-level scores for one retest attempt.

Column	Type	Null	Notes
id PK	UUID	no	
retest_attempt_id FK→retest_attempts.id UQ	UUID	no	
bloom_level UQ	enum bloom_level	no	
before	int	yes	
after	int	yes	

UNIQUE(retest_attempt_id, bloom_level)

topic_performance

Before/after mastery on a specific topic, linking a voice test's finding to the homework that targeted it.

Column	Type	Null	Notes
id PK	UUID	no	
test_id FK→voice_tests.id	UUID	no	
homework_id FK→homework.id	UUID	no	
topic_label	Text	no	
before_percent	int	yes	
after_percent	int	yes	

I. Leaderboard

student_points

A student's cumulative gamification points. 1:1 with users. Rank is computed at query time via a window function — never stored.

Column	Type	Null	Notes
student_id PK FK→users.id	UUID	no	PK is the FK itself — enforces 1:1
points	int	no	default 0

J. Reporting

report_configurations

A saved custom report definition (dimension/metrics/filters).

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
owner_id FK→users.id	UUID	no	
school_id FK→schools.id	UUID	yes	null = cross-tenant Super Admin report
name	str	no	
dimension	str	no	app-validated, not a DB enum
metrics	JSONB (list[str])	no	
filters	JSONB (dict)	no	default {}

report_shares

Sharing a saved report configuration with another school.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
report_configuration_id FK→report_configurations.id	UUID	no	
shared_by FK→users.id	UUID	no	
shared_with_school_id FK→schools.id	UUID	no	
access_level	str	no	default "VIEW" ("VIEW" or "VIEW_EXPORT")
expires_at	timestampz	yes	

K. Support

support_tickets

A school's support request to Super Admin.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
school_id FK→schools.id	UUID	no	
created_by FK→users.id	UUID	no	
subject	Text	no	
description	Text	no	
status	enum ticket_status	no	default OPEN (OPEN, IN_PROGRESS, RESOLVED)

support_ticket_comments

Threaded comments on a support ticket.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
ticket_id FK→support_tickets.id	UUID	no	
author_id FK→users.id	UUID	no	
body	Text	no	

L. Import Jobs

import_jobs

A bulk-import run (teacher invites, student creation, or custom questions) with row-level success/error counts.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
school_id FK→schools.id	UUID	no	
initiated_by FK→users.id	UUID	no	
job_type	enum import_job_type	no	TEACHER_INVITE, STUDENT_CREATE, CUSTOM_QUESTION
filename	str	no	
row_count	int	no	
created_count	int	no	
error_count	int	no	

M. AI Assistant (Teacher Chat)

assistant_messages

Chat history between a teacher and the AI teaching assistant.

Column	Type	Null	Notes
id PK	UUID	no	
created_at	timestampz	no	
teacher_id FK→users.id	UUID	no	
role	enum chat_role	no	user, assistant
text	Text	no	